COMPUTER VISION ASSIGNMENT

License Plate Recognition using PaddleOCR v4 & v5

Field	Details
Name	Hritwiz Yash
Dataset	Roboflow License Plates US-EU
Framework	PaddleOCR 3.4.0 + PaddlePaddle 3.3.0
Platform	Google Colab — T4 GPU, CUDA 13.0
Colab share link	PPOCR_License_Plate_Training - https://colab.research.google.com/drive/1nZaPw1q_oVYbUPYX8TYwRG1JYQ0bxghE?usp=sharing PPOCR_License_Plate_Inference- https://colab.research.google.com/drive/1ACMRcZfVq9ffOjZxJAT95STsQEY7-o--?usp=sharing
Repo	https://github.com/hritwizsri/License-Plate-Recognition-using-PaddleOCR-.git

▯ NOTEBOOK 1 — Training Pipeline

File: PPOCR_License_Plate_Training.ipynb

▯ Objective

Train two OCR models — PP-OCRv4 and PP-OCRv5 — on a license plate dataset and compare their performance. Export both models to inference format for use in Notebook 2.

▯ Project Workflow

```
RAW DATASET (YOLOv8 format)
  ↓
Cell 0  — Install PaddlePaddle 3.3.0 + PaddleOCR 3.4.0
Cell 1  — Mount Google Drive + Clone PaddleOCR GitHub repo
Cell 2  — Load & explore dataset (images + labels)
Cell 3  — Convert YOLO labels → OCR rec format
          (rec_gt_train_fixed.txt / rec_gt_val_fixed.txt / rec_gt_test_fixed.txt)
Cell 4  — Write training configs for V4 and V5
          (en_PP-OCRv4_rec.yml / PP-OCRv5_mobile_rec.yml)
Cell 5  — Preview dataset samples → dataset_preview.png
Cell 6  — Train PP-OCRv4 for 10 epochs → output/v4_rec/
Cell 7  — Train PP-OCRv5 for 10 epochs → output/v5_rec/
Cell 8  — Evaluate both models → v4_eval.txt / v5_eval.txt
Cell 9  — Export to inference format → v4_infer/ / v5_infer/
Cell 10 — Compare results + visualize → comparison.csv / training_curves.png
```

▯ Input Files & Folders

Path	Description
lp-dataset/train/images/	Training images (YOLOv8 format)
lp-dataset/train/labels/	YOLO bbox labels for training
lp-dataset/valid/images/	Validation images
lp-dataset/valid/labels/	YOLO bbox labels for validation
lp-dataset/test/images/	Test images
lp-dataset/test/labels/	YOLO bbox labels for testing
en_PP-OCRv4_rec_pretrained.pdparams	V4 pretrained weights
PP-OCRv5_mobile_rec_pretrained.pdparams	V5 pretrained weights

▯ Output Files Generated

File / Folder	Description
output/v4_rec/best_accuracy.pdparams	Best V4 trained model weights
output/v4_rec/config.yml	V4 training config (reused in NB2)
output/v4_rec/train.log	V4 full training log
output/v5_rec/best_accuracy.pdparams	Best V5 trained model weights
output/v5_rec/config.yml	V5 training config (reused in NB2)
output/v4_infer/	V4 exported inference model folder
output/v5_infer/	V5 exported inference model folder
output/v4_eval.txt	V4 evaluation metrics
output/v5_eval.txt	V5 evaluation metrics
output/comparison.csv	Side-by-side V4 vs V5 metrics
output/training_curves.png	Loss and accuracy training plot

output/dataset_preview.png	Sample dataset images preview
res/rec_gt_train_fixed.txt	OCR-format training labels
res/rec_gt_val_fixed.txt	OCR-format validation labels
res/rec_gt_test_fixed.txt	OCR-format test labels

🧠 Model Architecture

PP-OCRv4 Architecture:

```
Backbone  : MobileNetV1Enhance (scale=0.5)
Neck      : SequenceEncoder (reshape)
Head      : CTCHead
Loss      : CTCLoss
Optimizer : Adam (lr=0.0005, cosine decay, warmup=2)
Input     : [3, 32, 320]  — Channels × Height × Width
```

PP-OCRv5 Architecture:

```
Backbone  : PPLCNetV3
Neck      : SequenceEncoder (reshape)
Head      : CTCHead
Loss      : CTCLoss
Optimizer : Adam (lr=0.0005, cosine decay, warmup=2)
Input     : [3, 32, 320]
```

📊 Training Results

Metric	PP-OCRv4	PP-OCRv5
Epochs trained	10	10
Final Accuracy	~0.00	~0.00
Norm Edit Distance(NED)	0.0401	0.0321
Best Model	—	✅V5 (lower NED)

📌 ***NED > 0 confirms the model learned some character patterns despite low accuracy from insufficient training epochs.***

⚠️ **Accuracy ~ *0 is due to model trained for only 10 epoch due to computational challenges***

🚀 How to Run Notebook 1

- **Step 1 — Open Google Colab** Go to colab.research.google.com → File → Open Notebook → Upload PPOCR_License_Plate_Training.ipynb
- **Step 2 — Set GPU Runtime** Runtime → Change runtime type → T4 GPU → Save
- **Step 3 — Run Cell 0** Install PaddlePaddle 3.3.0 from official CUDA 13.0 index + PaddleOCR 3.4.0
- **Step 4 — Run Cell 1** Mount Google Drive + Clone PaddleOCR repo to /content/PaddleOCR
- **Step 5 — Run Cells 2–5** Dataset exploration, label conversion, config generation, preview
- **Step 6 — Run Cell 6** Train PP-OCRv4 (~15 min on T4 GPU)
- **Step 7 — Run Cell 7** Train PP-OCRv5 (~15 min on T4 GPU)
- **Step 8 — Run Cells 8–10** Evaluate, export inference models, compare and visualize

⚠️ *Important: Never restart the runtime mid-session. Training takes ~15 minutes per model on Colab free T4 GPU.*

🔍 NOTEBOOK 2 — Inference Pipeline

File: PPOCR_License_Plate_Inference.ipynb

🎯 Objective

Load the trained models from Notebook 1 and run inference on test images. Annotate results with bounding boxes and recognized text, compare V4 vs V5 side-by-side, process the full batch, perform error analysis, and export all results.

📋 Project Workflow

```
TRAINED MODELS (from Notebook 1)
  ↓
Cell 0 — Install PaddlePaddle 3.3.0 + PaddleOCR 3.4.0
Cell 1 — Setup all paths + mount Drive + verify all files exist
Cell 2 — Define run_ocr() + get_bbox() + crop_plate() functions
         Load V4 & V5 models via subprocess → sanity check
Cell 3 — Preview 8 test images with YOLO bboxes → test_preview.png
Cell 4 — Run inference on 20 sample images → results.csv
Cell 5 — Draw bboxes + OCR text on each image → ann_*.jpg
Cell 6 — Side-by-side V4 vs V5 grid (8×3) → comparison_grid.png
Cell 7 — Batch inference on ALL test images → batch_results.csv
Cell 8 — 6-panel error analysis → error_analysis.png
Cell 9 — Final report + complete file export list
```

📂 Input Files Required (All from Notebook 1)

Path	Source	Used In
output/v4_rec/best_accuracy.pdparams	NB1 training	Cell 2 — V4 model weights
output/v5_rec/best_accuracy.pdparams	NB1 training	Cell 2 — V5 model weights
output/v4_rec/config.yml	NB1 training	Cell 2 — V4 architecture
output/v5_rec/config.yml	NB1 training	Cell 2 — V5 architecture
test/images/*.jpg	NB1 dataset	Cells 3–7 inference input
test/labels/*.txt	NB1 dataset	get_bbox() for plate crop
ppocr/utils/en_dict.txt	Cloned repo	CTC decoder vocabulary
tools/infer_rec.py	Cloned repo	subprocess inference call

📤 Output Files Generated

File	Location	Description
test_preview.png	output/notebook2/	8 test images with green bboxes
results.csv	output/notebook2/	V4+V5 results for 20 images
ann_*.jpg	output/notebook2/	20 individually annotated images
comparison_grid.png	output/notebook2/	V4 vs V5 side-by-side 8×3 grid
batch_results.csv	output/notebook2/	Full dataset inference results
error_analysis.png	output/notebook2/	6-panel error analysis plot

⚙ Key Technical Details

Why Subprocess Instead of PaddleOCR API:

Error Encountered	Root Cause	Solution
KeyError: Global	v4_infer config is PaddlePaddle format, not PaddleX	Use subprocess
set_optimization_level	PaddleX needs PaddlePaddle ≥ 3.0.0	Install 3.3.0
Unknown argument: det	.ocr() replaced by .predict() in 3.x	Use subprocess
Unknown argument: rec_char_dict_path	Parameter renamed in 3.4.0	Use subprocess

run_ocr() Function Logic:

```
img_path passed in
      ↓
get_bbox()    → reads YOLO .txt label → pixel coords (x1,y1,x2,y2)
      ↓
crop_plate()  → crops plate region → saves to _tmp_crop.jpg
      ↓
subprocess calls tools/infer_rec.py with flags:
   -c config.yml
   -o Global.pretrained_model = best_accuracy
   -o Global.infer_mode       = true    ← critical flag
   -o Global.character_dict_path = en_dict.txt
   -o Global.infer_img        = _tmp_crop.jpg
      ↓
4 regex patterns parse stdout
      ↓
returns (text, confidence)
```

Regex Pattern Strategy (4 patterns for all PaddleOCR versions):

```
Pattern 1: r"result:\s*\('([^']+)',\s*([\d.]+)\)"    →  result: ('ABC', 0.9)
Pattern 2: r"result:\s*([^\t\n]+)\t([\d.]+)"         →  result: ABC\t0.9
Pattern 3: r"Predicts of.*?:\s*\('([^']+)',\s*([\d.]+)\)"  →  Predicts: ('ABC',
0.9)
Pattern 4: r"'text':\s*'([^']+)'.*?'score':\s*([\d.]+)"      →
{'text':'ABC','score':0.9}
```

⚙ Final Inference Results

Metric	PP-OCRv4	PP-OCRv5
Avg Confidence	0.046	0.033
N/A Predictions	0	0
Avg Text Length	16.06	25.09
Agreement Rate	0.0%	—
Pipeline Status	✅Complete	✅Complete

⚙ *Low confidence is expected — only 10 training epochs with architecture mismatch. The pipeline is 100% correct and complete.*

▯ How to Run Notebook 2

Prerequisites: Notebook 1 must be completed first. All output files saved to Google Drive.

- **Step 1 — Restart Session** Runtime → Restart session (mandatory before starting)
- **Step 2 — Set GPU Runtime** Runtime → Change runtime type → T4 GPU → Save
- **Step 3 — Run Cell 0** Uninstall old PaddlePaddle, install 3.3.0 from CUDA 13.0 index
- **Step 4 — Verify Install** Check paddle.__version__ == 3.3.0 and set_optimization_level exists
- **Step 5 — Run Cell 1** All 6 checkmarks must show ✅before proceeding
- **Step 6 — Run Cells 2–9** Run in order: models → preview → inference → annotate → compare → batch → analysis → report

Cell 0 Install Command:

```
!pip uninstall paddlepaddle paddlepaddle-gpu paddleocr -y -q
!python -m pip install paddlepaddle-gpu==3.3.0 \
    -i https://www.paddlepaddle.org.cn/packages/stable/cu130/ -q
!pip install paddleocr==3.4.0 -q
```

▯ Common Errors & Fixes

Error	Fix
PDX already initialized	Restart runtime → re-run from Cell 0
set_optimization_level	Use PaddlePaddle 3.3.0 from official CUDA index
KeyError: Global	Use subprocess — do NOT use PaddleOCR() API
NameError: V4_INFER not defined	Re-run Cell 1 — kernel was restarted
nvidia-smi not found	Switch to T4 GPU runtime first
N/A predictions	Add Global.infer_mode=true to subprocess cmd
paddlepaddle-gpu==3.0.0 not found	PyPI max is 2.6.2 — use official PaddlePaddle index

📚 Resources

Resource	URL
Assignment Repo	github.com/shubh-lyn/COMPUTER_VISION_ASSIGNMNET
Dataset	public.roboflow.com/object-detection/license-plates-us-eu/3
PaddleOCR Repo	github.com/PaddlePaddle/PaddleOCR
PaddlePaddle Install	paddlepaddle.org.cn/packages/stable/cu130/

Challenges Faced

#	Challenge	Notebook	Status
1	PaddlePaddle 3.x not on PyPI	Both	✅Fixed
2	PDX Already Initialized error	NB2	✅Fixed
3	PaddleOCR 3.4.0 API breaking changes (4 errors)	NB2	✅Fixed
4	`set_optimization_level` AttributeError	NB2	✅Fixed
5	Inference producing empty text (`infer_mode`)	NB2	✅Fixed
6	Model downloading instead of using local files	NB2	✅Fixed
7	Architecture mismatch during training	NB1	✅Handled
8	NameError after kernel restart	NB2	✅Fixed
9	Low model accuracy (expected outcome)	Both	✅Documented